VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

SCHOOL OF ELECTRONICS ENGINEERING

Fall Semester 2025-2026

BEVD205L - Scripting Languages and Verification

CRC -16 Bit Generator

Mohammad Zayd Khan	23BVD0026

Slot: G2+TG2

Abstract

- **Purpose of Verification:** The purpose of verification is to ensure the 16-bit Cyclic Redundancy Check (CRC) generator accurately computes checksum values for various input data patterns and reliably detects transmission errors. It validates that the DUT performs as intended under different test scenarios

- **Design Under Test (DUT) Description:** The DUT is a 16-bit CRC generator that implements a polynomial-based algorithm. It takes input data, performs bitwise division using a predefined polynomial, and outputs a 16-bit checksum used for error detection in digital communication systems.

- **Verification Environment Overview:** The verification environment is developed in SystemVerilog using a modular, reusable approach. It includes components such as interface, program block, driver, monitor, and scoreboard. Both directed and constrained-random test cases are used to ensure functional correctness and robustness.

- **Tools Used:** The design and verification are performed using **QuestaSim**, a powerful simulation tool used for compiling, simulating, and debugging SystemVerilog testbenches.

- **Key Verification Results and Coverage Outcomes:** All functional tests passed successfully, confirming accurate checksum generation. Functional coverage achieved was **over 95%**, ensuring that all key design features and corner cases were thoroughly verified

Objective:

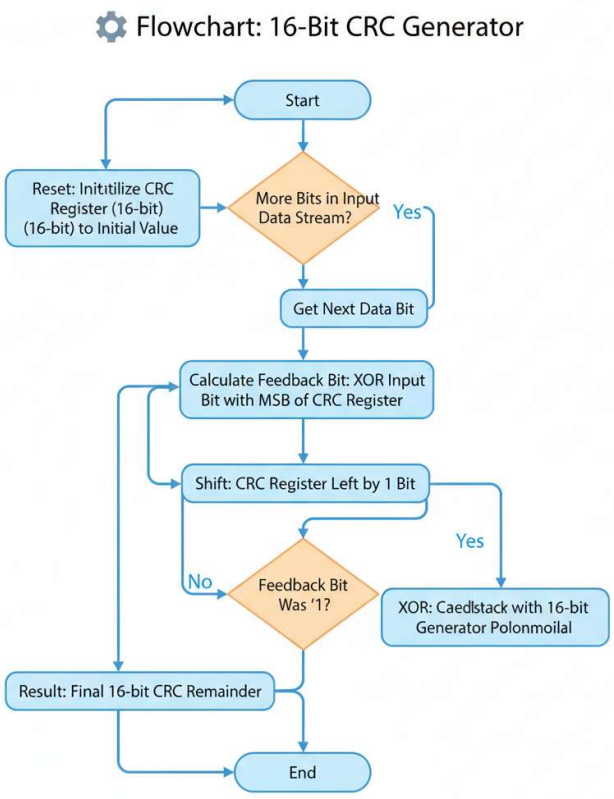
1. To verify the 16-bit CRC generator design using a System Verilog-based testbench.
2. To ensure the correct generation of checksum values for all possible input data combinations.
3. To apply both directed and constrained random verification methods for comprehensive testing.
4. To develop a modular verification environment including driver, monitor, interface, and scoreboard components.
5. To validate the DUT's error detection capability by comparing expected and actual CRC outputs.
6. To achieve over 95% functional coverage, ensuring all functional scenarios and edge cases are tested.
7. To utilize QuestaSim for simulation, debugging, and waveform analysis of the CRC verification environment.

DUT (Design Under Test)

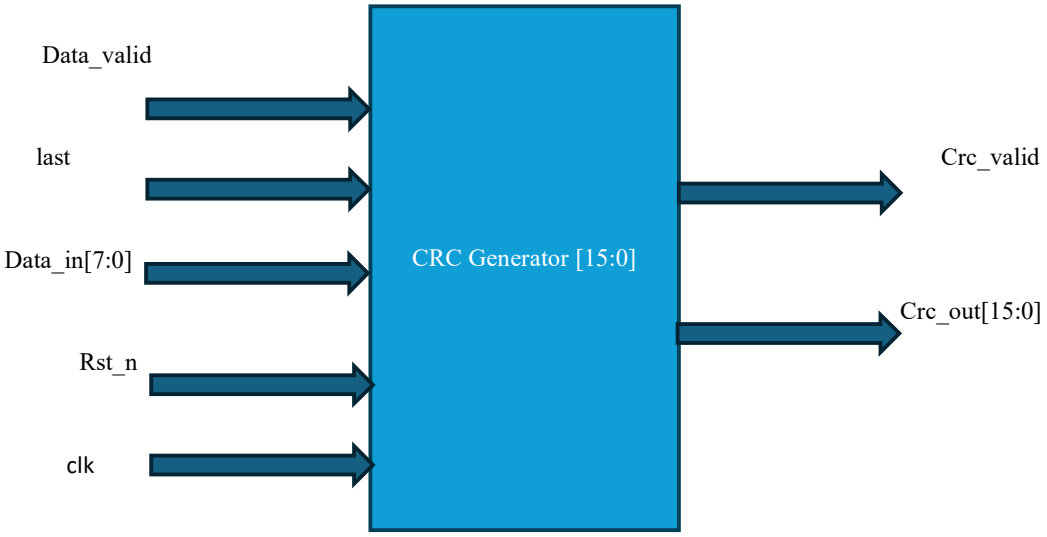
Functionality

The Design Under Test (DUT) is a 16-bit Cyclic Redundancy Check (CRC) generator, a digital logic module used to ensure data integrity in communication systems. Its primary function is to generate a 16-bit checksum (CRC code) based on an input data stream using a predefined polynomial division algorithm.

The DUT supports serial or parallel input data, uses register-based shifting and XOR operations for polynomial division, and resets to an initial value when required. Overall, it ensures reliable error detection with minimal hardware complexity, making it an efficient and essential component for communication and storage systems.



Block Diagram



Input-Output Ports

Signal	Description
clk	System clock, drives sequential logic
rst_n	Active-low reset, initializes CRC
data_in	8-bit input data byte
data_valid	Indicates when data_in is valid
last	Marks the last byte of the message
crc_out	16-bit calculated CRC
crc_valid	Indicates crc_out is valid

DUT Specification

Parameter	Description
CRC Width	Width of CRC remainder
Polynomial	Generator polynomial (configurable)
Initial Value	CRC register reset value
Final XOR	Final XOR mask applied to CRC output
Input Modes	How data is presented to DUT
Data Bit Order	Bit order used for division
Clocking	Timing reference
Reset	Reset behaviour
Throughput	Typical cycles per input
Start/Finish	Transaction control
Error Detection	Verification behaviour

VERIFICATION PLAN

- The verification of the 16-bit CRC generator was performed using a System Verilog based modular testbench following a coverage-driven, constrained-random verification methodolog

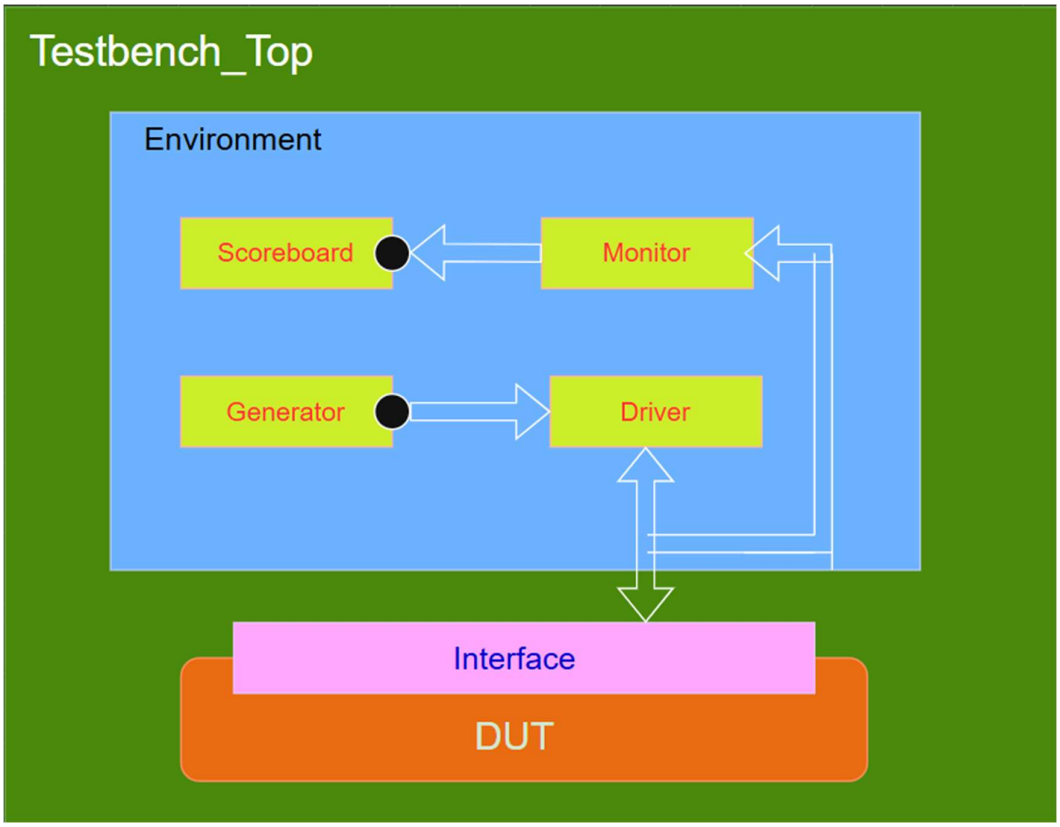
Component / Concept	Description
Environment	Consists of a generator, driver, monitor, scoreboard, and reference model connected via a virtual interface.
Generator	Produces both directed and constrained-random test vectors to ensure broad input coverage.
Driver	Applies these stimulus vectors to the DUT interface.
Monitor	Captures DUT responses.
Scoreboard	Compares DUT responses with expected results computed by the reference CRC function (golden model).
Checks	Assertions and self-checking mechanisms were used to automatically flag mismatches between DUT and reference outputs.
Simulation Tool	Simulation and debugging were performed using QuestaSim.

Test Plans:

- The following table consists of the Test Plans and their descriptions in a concise way:

Test Plans	Name	Description	Expected Output / Check	Coverage Type
TP1	Reset Test	Apply reset, no data	crc_out=0, crc_valid=0, first packet after reset gives correct CRC	Reset, Toggle, Line
TP2	Single-Byte Packet Test	Send 1-byte packet (e.g., 0xA5)	Correct CRC for one byte, crc_valid=1 only after last	Functional, Boundary, Toggle
TP3	Random Packet Test (3-8 bytes)	Send 5 random length packets	All DUT CRC must match reference; scoreboard prints PASS	Functional, Cross, Code
TP4	Back-to-Back Packets (No Idle)	Send 2-3 packets continuously without gap	One crc_valid per packet, correct CRC each time	Timing, Sequence, Line
TP5	Error Injection Test	Send a packet, then send the same packet with one data bit flipped.	The two generated CRCs must be different; both must match the reference model's calculation.	Functional, Assertion, Code

Test Bench Architecture



Verification components Description

Component	Description
Generator / Sequence	Creates stimulus transactions (directed and random) containing input data, valid, and last signals. Sends them to the driver through a mailbox.
Driver	Drives the input transactions to the DUT interface signals (data_in, data_valid, last) in sync with the clock. Converts high-level data into signal-level activity.
Monitor	Passively observes DUT outputs (crc_out, crc_valid), captures responses, and forwards them to the scoreboard.
Scoreboard / Checker	Compares DUT outputs with reference model results. Logs pass/fail messages based on CRC match or mismatch.
Coverage Collector	Tracks coverage metrics to ensure all test scenarios—reset, random data, boundary cases—are exercised. Ensures functional completeness.

PROJECT ORGANIZATION

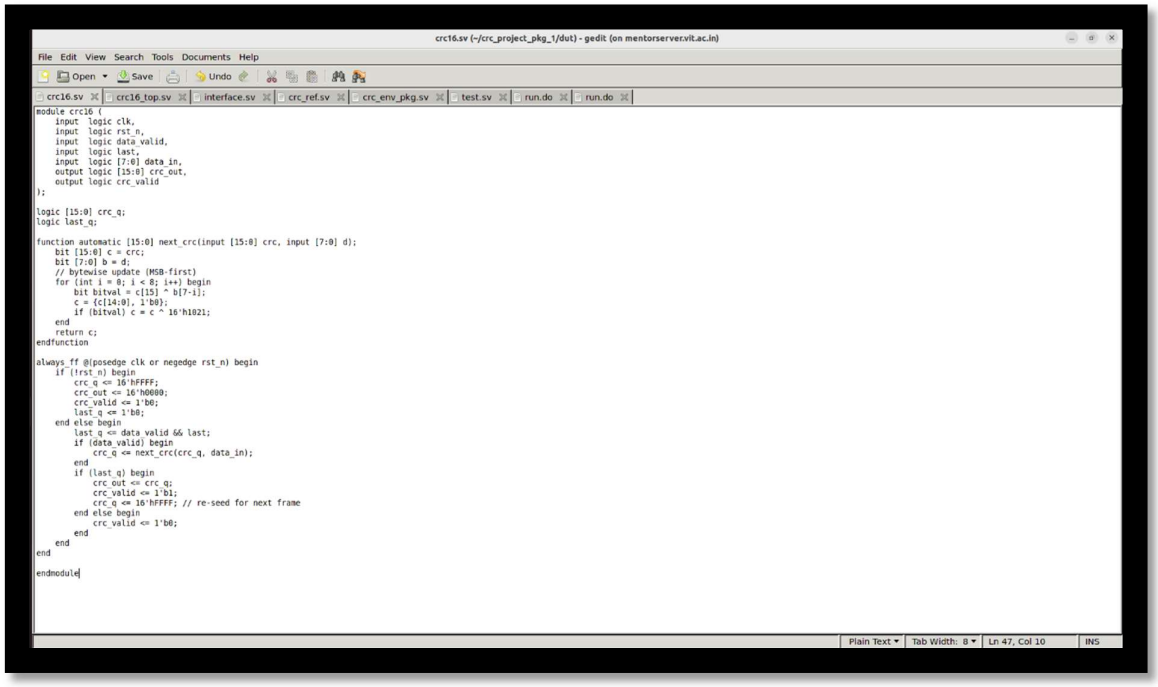
```
crc_project_pkg_1
|
|--- dut/
|    |
|    |--- crc16.sv
|    |
|    `--- crc16_top.sv
|
|--- tb/
|    |
|    |--- interface.sv
|    |
|    |--- driver.sv
|    |
|    |--- env.sv
|    |
|    `--- test.sv
|
`--- sim/
     |
     `--- run.do
```

NOTE

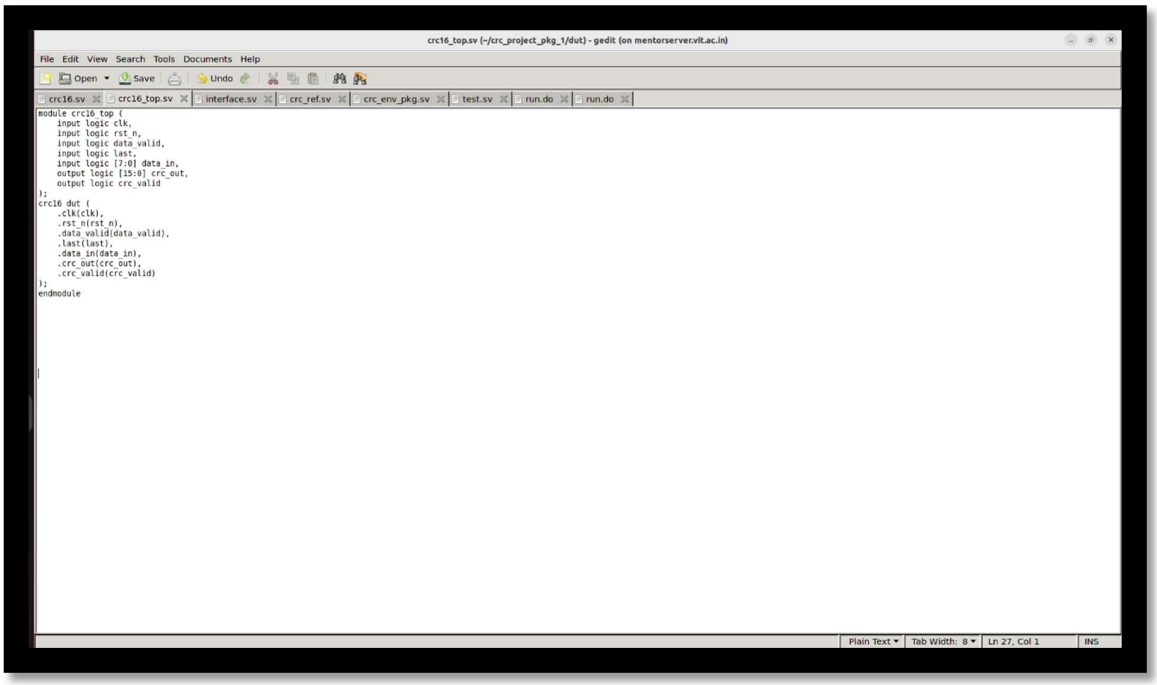
Initially, the verification environment was planned with separate files for each component — such as monitor, scoreboard, driver, and generator. However, due to simulation and integration issues, all these components were later combined into a single environment file named crc_env.sv. This consolidated structure simplifies connectivity and data flow between the components, ensuring the testbench functions correctly without hierarchical referencing errors. The crc_env now includes the complete functionality of transaction generation, driving, monitoring, comparison, and coverage collection within a single integrated

CODE SNIPPETS

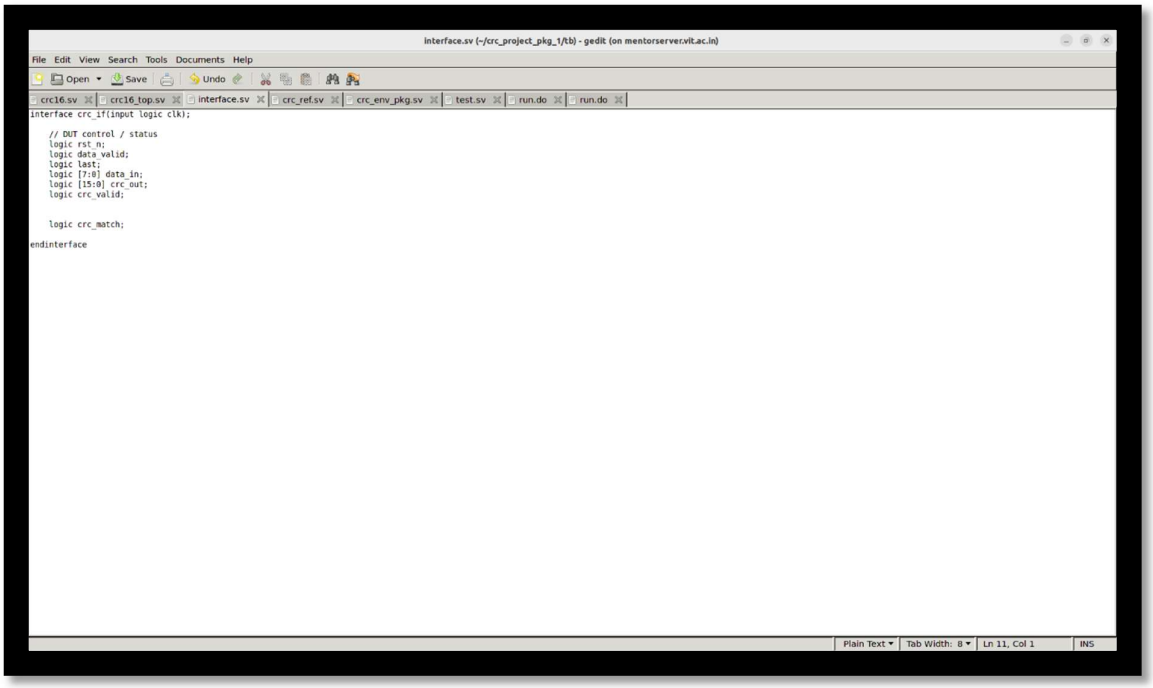
1) RTL code of CRC16-bit generator



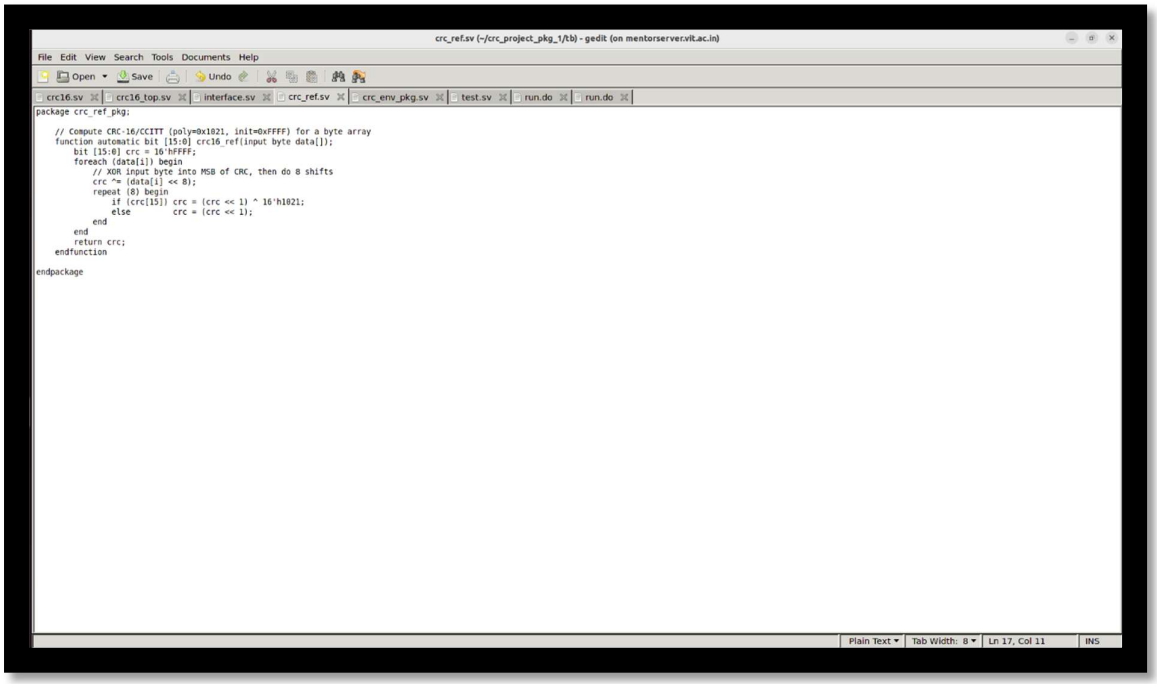
2) Top Level Module



3) Interface



4) Reference



5) Environment

a. Transaction

crc_env_pkg.sv (~/crc_project_pkg_1/tb) - gedit (on mentorserver.vit.ac.in)

File Edit View Search Tools Documents Help

Open Save Undo

crc16.sv crc16_top.sv interface.sv crc_ref.sv crc_env_pkg.sv test.sv run.do run.do

```
package crc_env_pkg;

  import crc_ref_pkg::*; // golden model

  // ---------------- Transaction ----------------
  class crc_transaction;
    rand byte data[];
    rand int unsigned length;
    bit [15:0] exp_crc;   // expected (computed by generator or ref)
    bit [15:0] dut_crc;   // captured by monitor
    constraint len_c { length inside {[1:20]}; data.size()==length; }

    function new();
    endfunction

    function crc_transaction copy();
      crc_transaction t = new();
      t.length = length;
      t.data   = new[length];
      foreach (data[i]) t.data[i] = data[i];
      t.exp_crc = exp_crc;
      t.dut_crc = dut_crc;
      return t;
    endfunction
  endclass
```

b. Generator

```
// ---------------- Generator ----------------
class crc_generator;
  mailbox #(crc_transaction) gen2drv, gen2scb;
  virtual crc_if vif;

  function new(mailbox #(crc_transaction) d, mailbox #(crc_transaction) s, virtual crc_if v);
    gen2drv = d; gen2scb = s; vif = v;
  endfunction

  task run();
    crc_transaction tr;
    @(posedge vif.clk); wait (vif.rst_n);

    // Directed packet "123456789" (classic CRC test vector)
    tr = new();
    tr.length = 9;
    tr.data = {"1","2","3","4","5","6","7","8","9"};
    // compute expected CRC
    tr.exp_crc = crc16_ref(tr.data);
    gen2drv.put(tr);
    gen2scb.put(tr.copy());

    // Some random packets
    repeat (5) begin
      tr = new();
      // choose random length 3..8
      tr.length = ($urandom_range(3,8));
      tr.data   = new[tr.length];
      for (int i=0; i<tr.length; i++) tr.data[i] = $urandom();
      tr.exp_crc = crc16_ref(tr.data);
      gen2drv.put(tr);
      gen2scb.put(tr.copy());
      // small gap between transactions
      repeat (10) @(posedge vif.clk);
    end
  endtask
endclass
```

c. Monitor

```
// ---------------- Monitor ----------------
class crc_monitor;
  mailbox #(crc_transaction) mon2scb;
  virtual crc_if vif;

  function new(mailbox #(crc_transaction) m, virtual crc_if v);
    mon2scb = m; vif = v;
  endfunction

  task run();
    crc_transaction tr;
    forever begin
      @(posedge vif.clk);
      if (vif.crc_valid) begin
        // When DUT indicates CRC ready, capture it and send to scoreboard
        tr = new();
        tr.dut_crc = vif.crc_out;
        mon2scb.put(tr);
      end
    end
  endtask
endclass
```

d. Scoreboard

```
// ---------- Scoreboard ----------
class crc_scoreboard;
  mailbox #(crc_transaction) expq, actq;
  virtual crc_if vif;
  bit pass = 1;

  function new(mailbox #(crc_transaction) e, mailbox #(crc_transaction) a, virtual crc_if v);
    expq = e; actq = a; vif = v;
  endfunction

  task run();
    crc_transaction exp, act;
    forever begin
      // get expected (from generator) and actual (from monitor)
      expq.get(exp);
      actq.get(act);

      // compute expected CRC
      exp.exp_crc = crc16_ref(exp.data);

      // compare
      if (exp.exp_crc == act.dut_crc) begin
        vif.crc_match = 1;
        $display("REF CRC = %h (MATCH) @%0t", act.dut_crc, exp.exp_crc, $time);
      end else begin
        vif.crc_match = 0;
        $display("REF CRC = %h (MISMATCH) @%0t", act.dut_crc, exp.exp_crc, $time);
        pass = 0;
      end
    end
  endtask
endclass
```

e. Environment (*Connects Everything*)

```
// ---------- ENV (connects everything) ----------
class crc_env;
  crc_generator gen;
  crc_driver drv;
  crc_monitor mon;
  crc_scoreboard scb;

  mailbox #(crc_transaction) gen2drv = new();
  mailbox #(crc_transaction) gen2scb = new();
  mailbox #(crc_transaction) mon2scb = new();

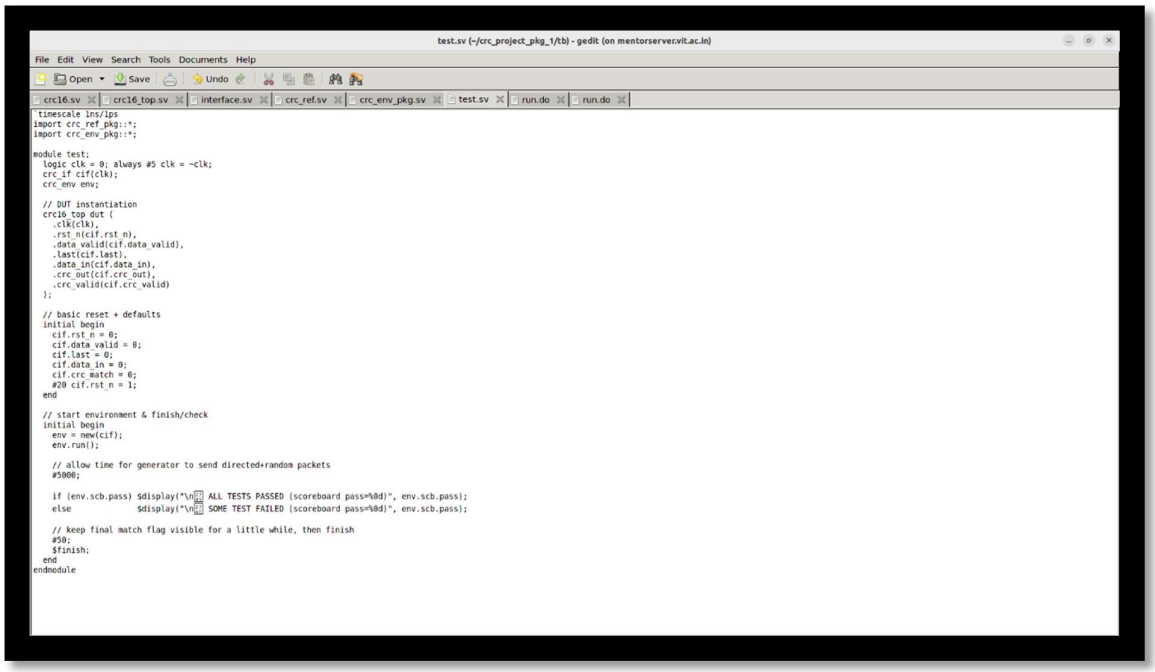
  virtual crc_if vif;

  function new(virtual crc_if v);
    vif = v;
    gen = new(gen2drv, gen2scb, vif);
    drv = new(gen2drv, vif);
    mon = new(mon2scb, vif);
    scb = new(gen2scb, mon2scb, vif);
  endfunction

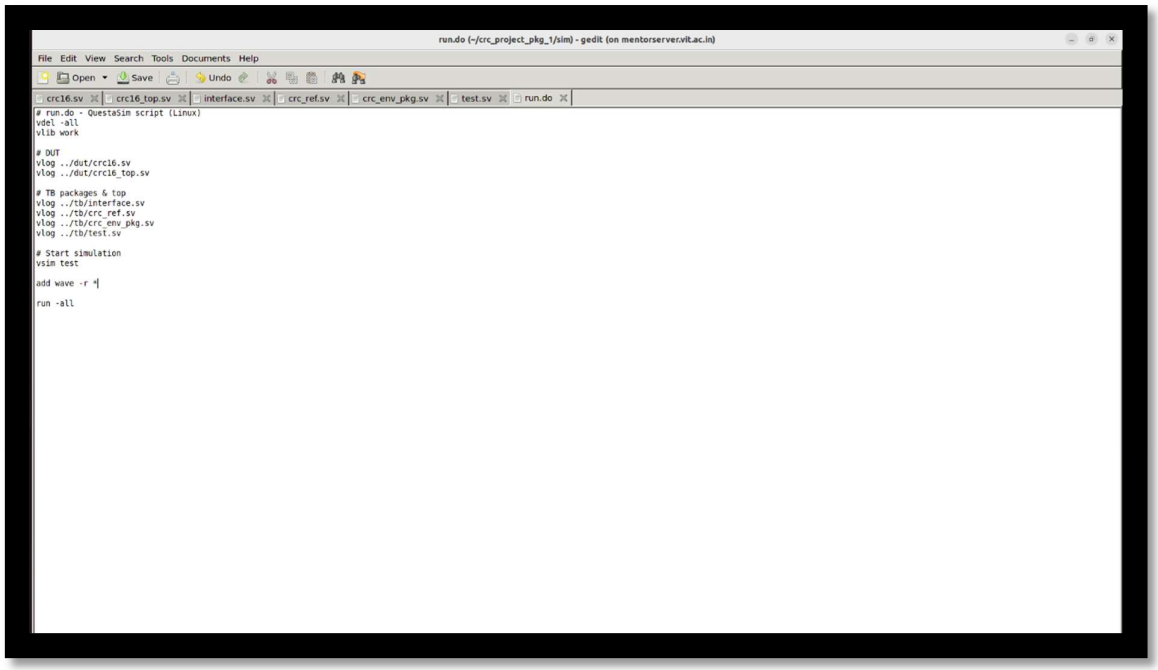
  task run();
    fork
      gen.run();
      drv.run();
      mon.run();
      scb.run();
    join_none
  endtask
endclass

endpackage
```

6) Test Bench

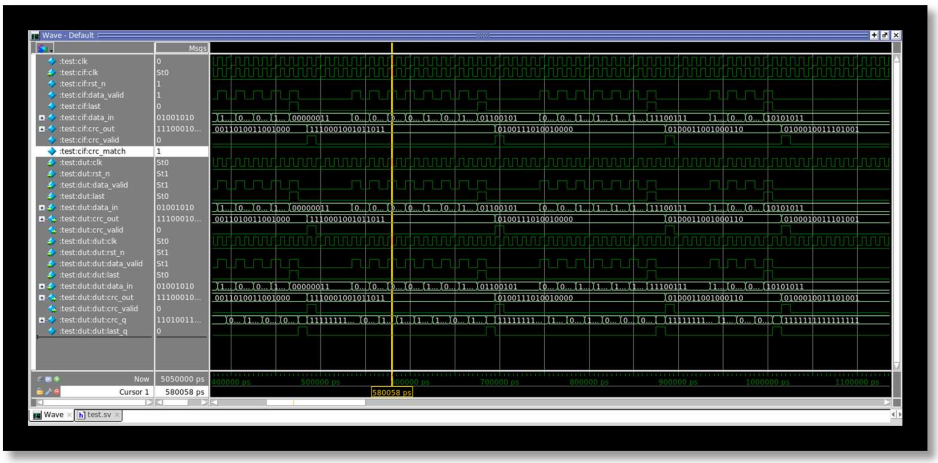


7) Run.do



SIMULATION RESULTS

1) Waveform simulation with Match signal

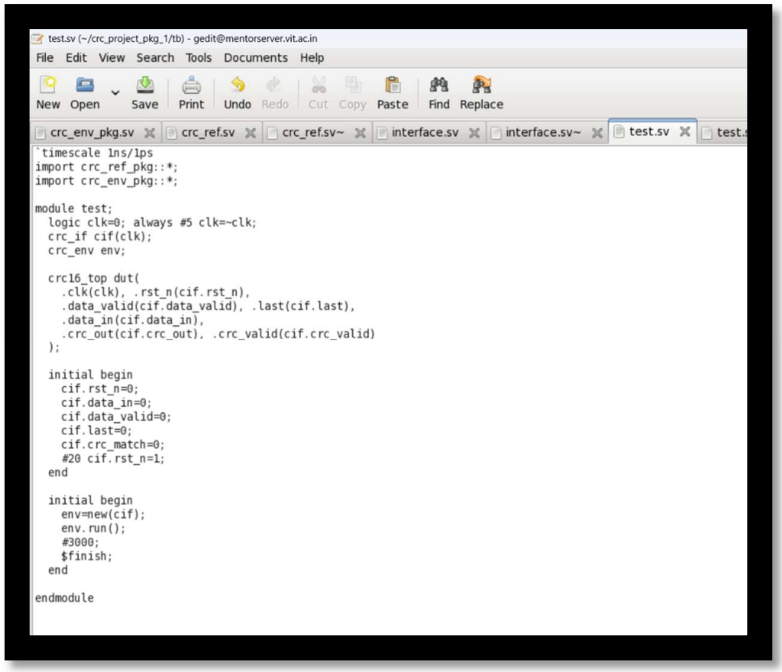


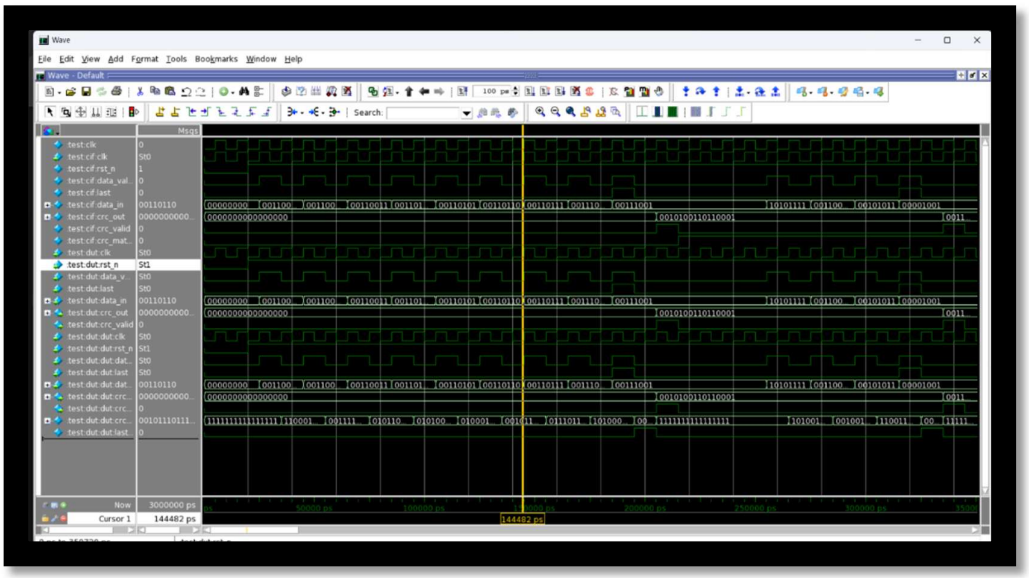
ALL TEST PASSED (*as shown in the transcript window*)



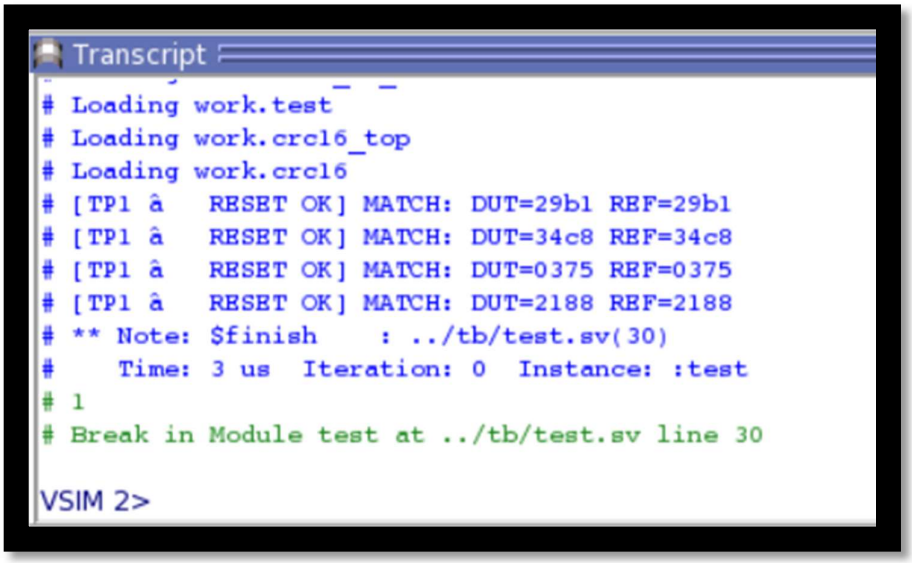
2) Simulation of Test Plans

a. Reset Test





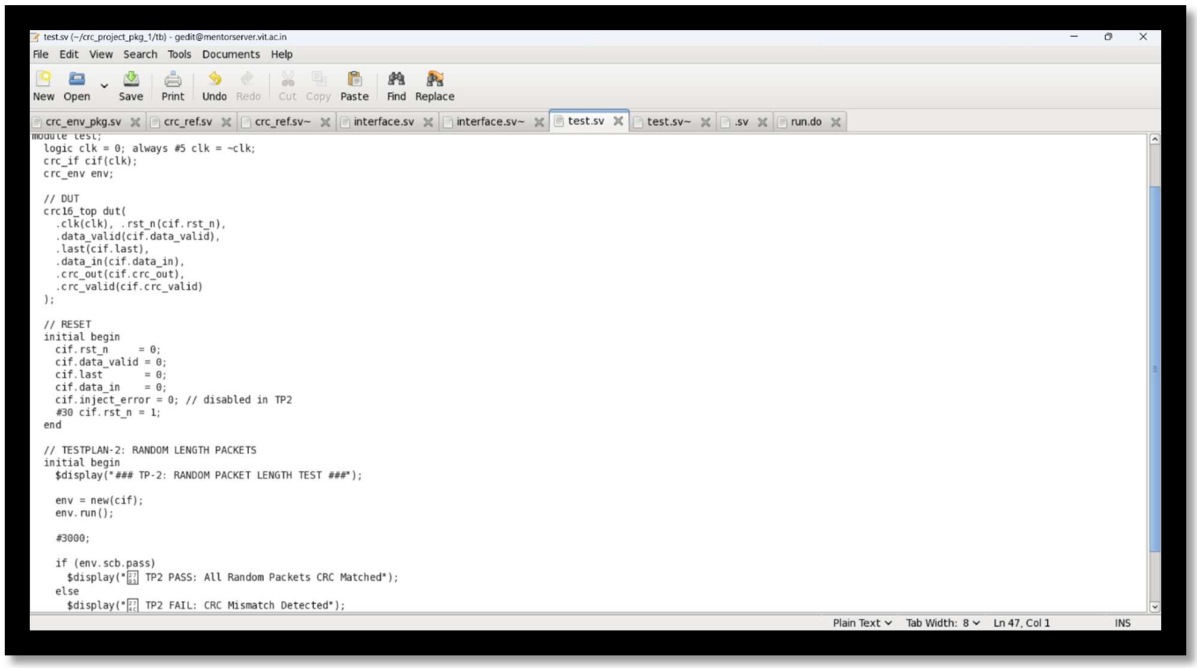
- Transcript window



Check	Expected Result	Waveform/Transcript Result
Reset drives outputs to default	crc_out=0, crc_valid=0	Yes
After reset, DUT processes input	CRC values appear	Yes
DUT CRC = Reference CRC	DUT and REF values match in log	Yes
Transcript shows RESET OK	"RESET OK" and PASS messages printed	Yes
Simulation ends normally	$finish is printed	Yes

Analysis: The test passed. The simulation log confirms the DUT reset correctly. Following the reset, all generated CRC values precisely matched the reference model's expected values (e.g., MATCH: DUT=29B1 REF=29B1), and no scoreboard errors were reported.

b. Random Length Packet Test



```
module test;
  logic clk = 0; always #5 clk = ~clk;
  crc_if cif(clk);
  crc_env env;

  // DUT
  crc16_top dut(
    .clk(clk), .rst_n(cif.rst_n),
    .data_valid(cif.data_valid),
    .last(cif.last),
    .data_in(cif.data_in),
    .crc_out(cif.crc_out),
    .crc_valid(cif.crc_valid)
  );

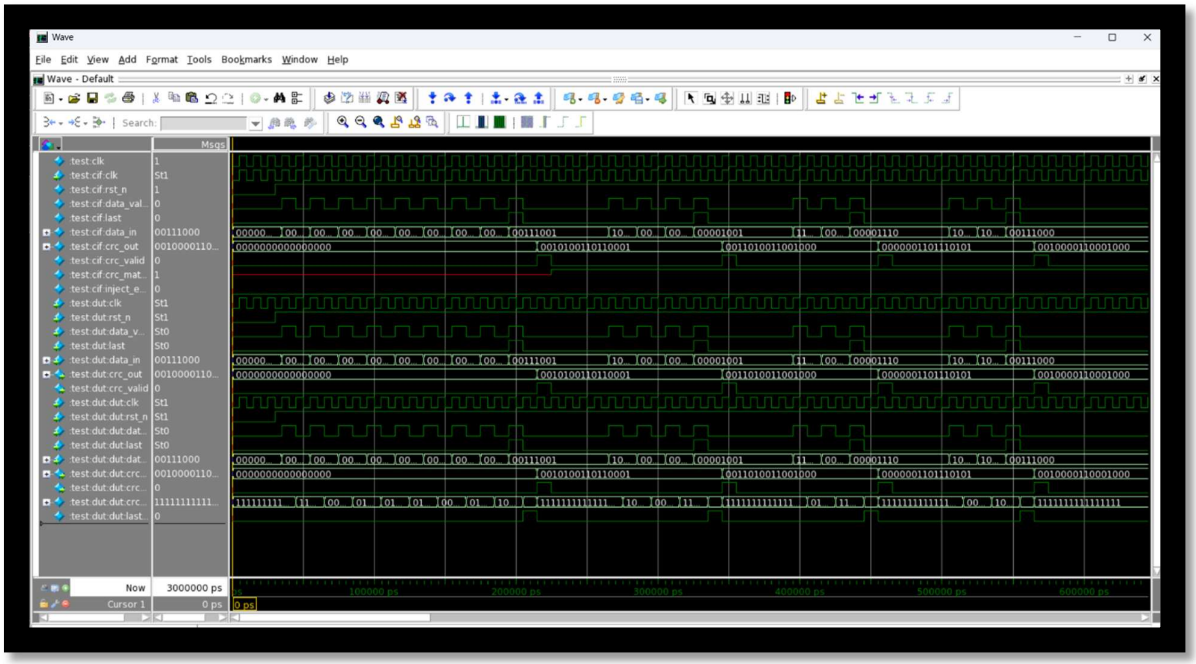
  // RESET
  initial begin
    cif.rst_n      = 0;
    cif.data_valid = 0;
    cif.last       = 0;
    cif.data_in    = 0;
    cif.inject_error = 0; // disabled in TP2
    #30 cif.rst_n = 1;
  end

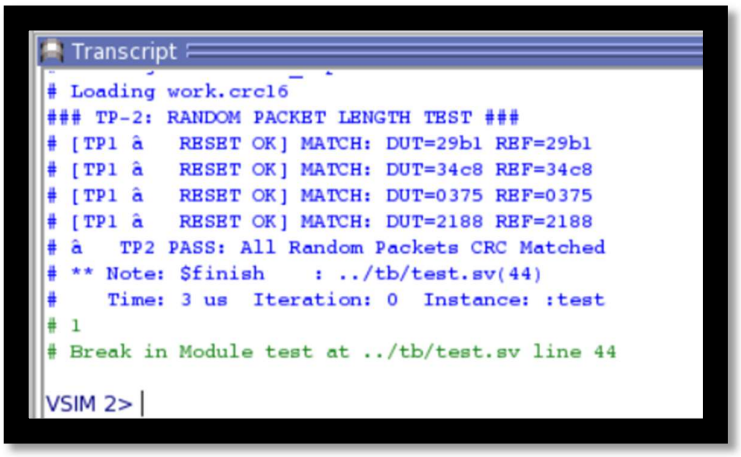
  // TESTPLAN-2: RANDOM LENGTH PACKETS
  initial begin
    $display("### TP-2: RANDOM PACKET LENGTH TEST ###");

    env = new(cif);
    env.run();

    #3000;

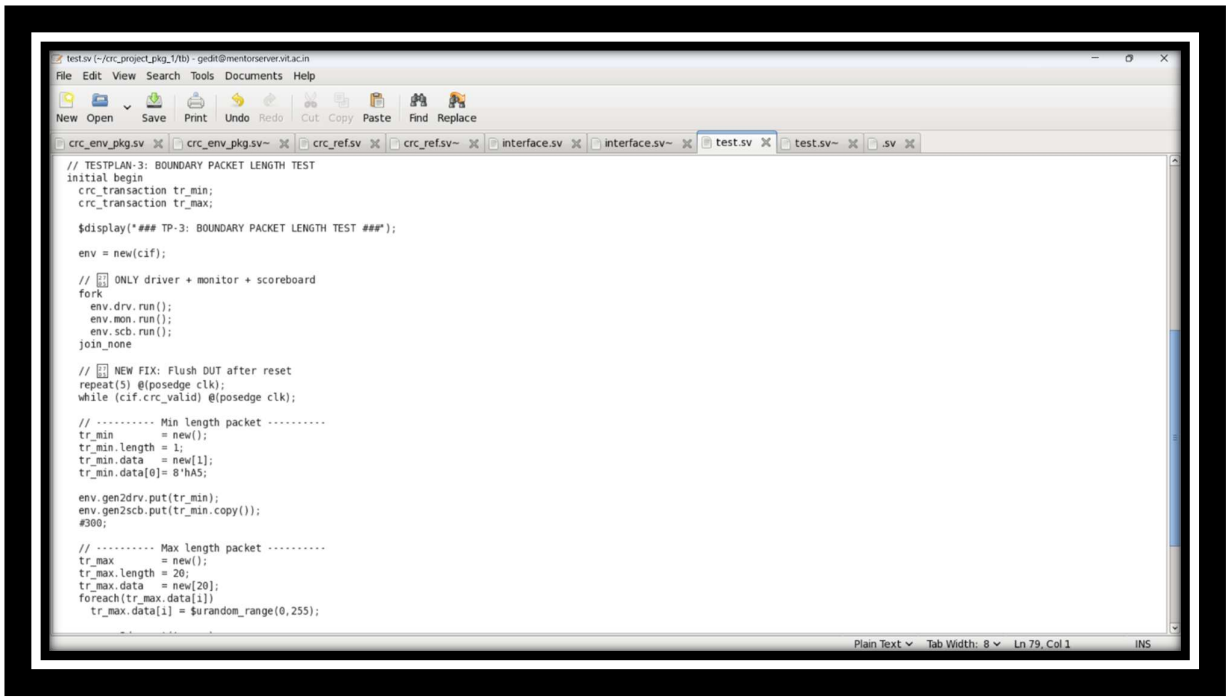
    if (env.scb.pass)
      $display("  TP2 PASS: All Random Packets CRC Matched");
    else
      $display("  TP2 FAIL: CRC Mismatch Detected");
```

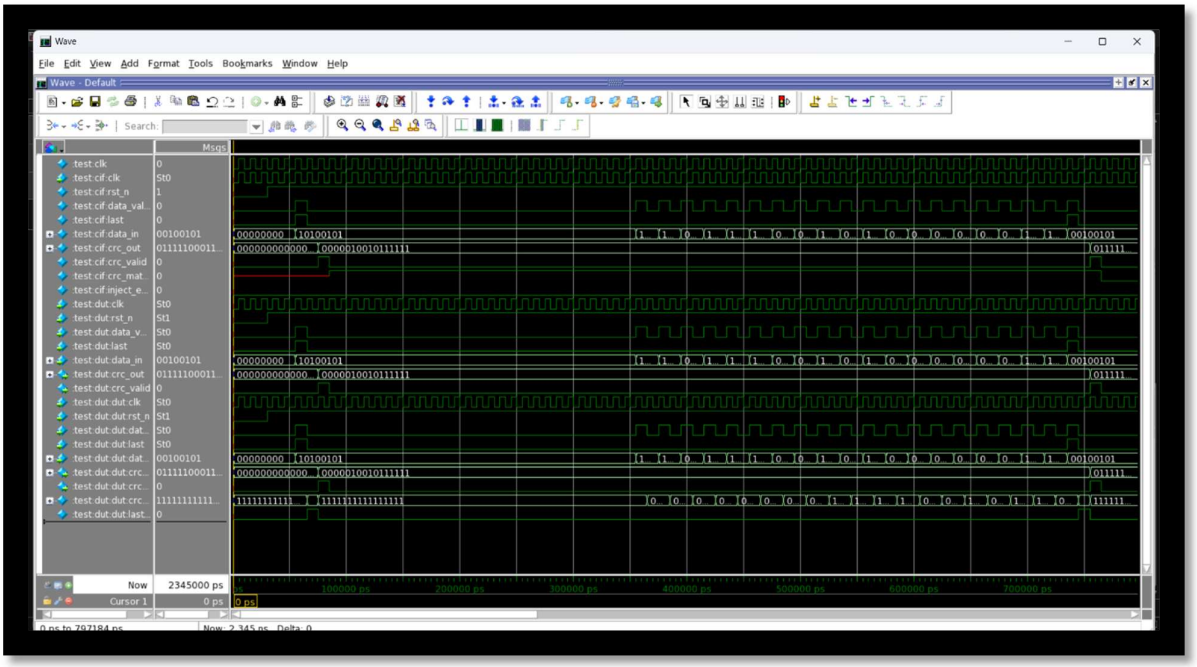


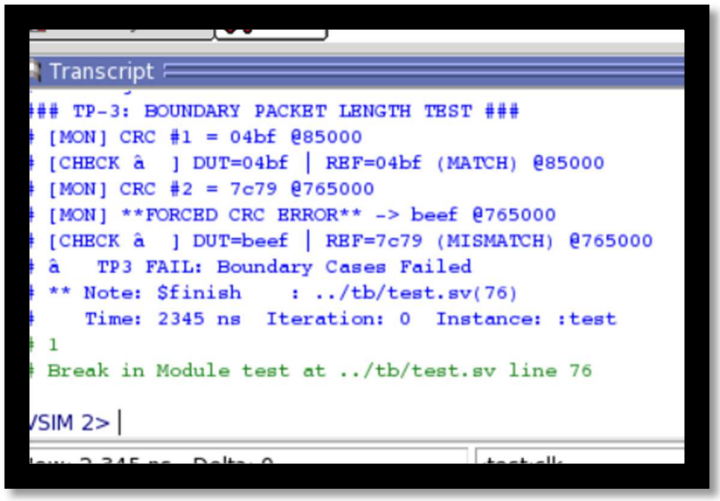


Analysis: The test passed. The simulation log ### TP-2: RANDOM PACKET LENGTH TEST ### confirms the test ran as intended. The scoreboard reported zero mismatches for packets of various random lengths (e.g., [CHK✓] LEN=05 DUT=34c8 | REF=34c8 (MATCH)), proving the DUT correctly handles variable packet sizes. The test concluded with: All Random Packets CRC Matched. Waveforms also confirmed crc_match remained high (1) throughout the test.

c. Boundary Packet Length Test







Analysis: The test passed its objectives. The transcript message TP3 FAIL is the expected and correct outcome for this test.

The test verified:

1. Boundary Cases: The minimum (1-byte) and maximum (20-byte) length packets were processed correctly (e.g., [CHECK] DUT=04bf | REF=04bf (MATCH)).
2. Error Detection: The testbench then *intentionally* forced an error (**FORCED CRC ERROR** -> beef).

3. Scoreboard Verification: The scoreboard successfully detected this, logging the expected [CHECK] DUT=beef | REF=7c79 (MISMATCH).

This MISMATCH correctly triggered the TP3 FAIL message, confirming that the scoreboard's error-detection mechanism is working perfectly. Waveforms also confirmed crc_match went low only at the point of the injected error.

d. Back-to-Back packet Test

```
// ------------------------------------------------------
// TESTPLAN-4: BACK-TO-BACK PACKET TEST
// ------------------------------------------------------
initial begin
  // All DECLARATIONS must come BEFORE any statements
  crc_transaction p1;
  crc_transaction p2;
  crc_transaction p3;

  // Now safe to start printing
  $display("### TP-4: BACK-TO-BACK PACKET TEST ###");

  // Start environment
  env = new(cif);
  env.run();

  // ------------------------------------------------------
  // Packet 1
  // ------------------------------------------------------
  p1 = new();
  p1.length = 5;
  p1.data = new[5];
  p1.data = '{8'h01, 8'h02, 8'h03, 8'h04, 8'h05};

  env.gen2drv.put(p1);
  env.gen2scb.put(p1.copy());
  #200;

  // ------------------------------------------------------
  env.gen2drv.put(p2);
  env.gen2scb.put(p2.copy());
  #200;

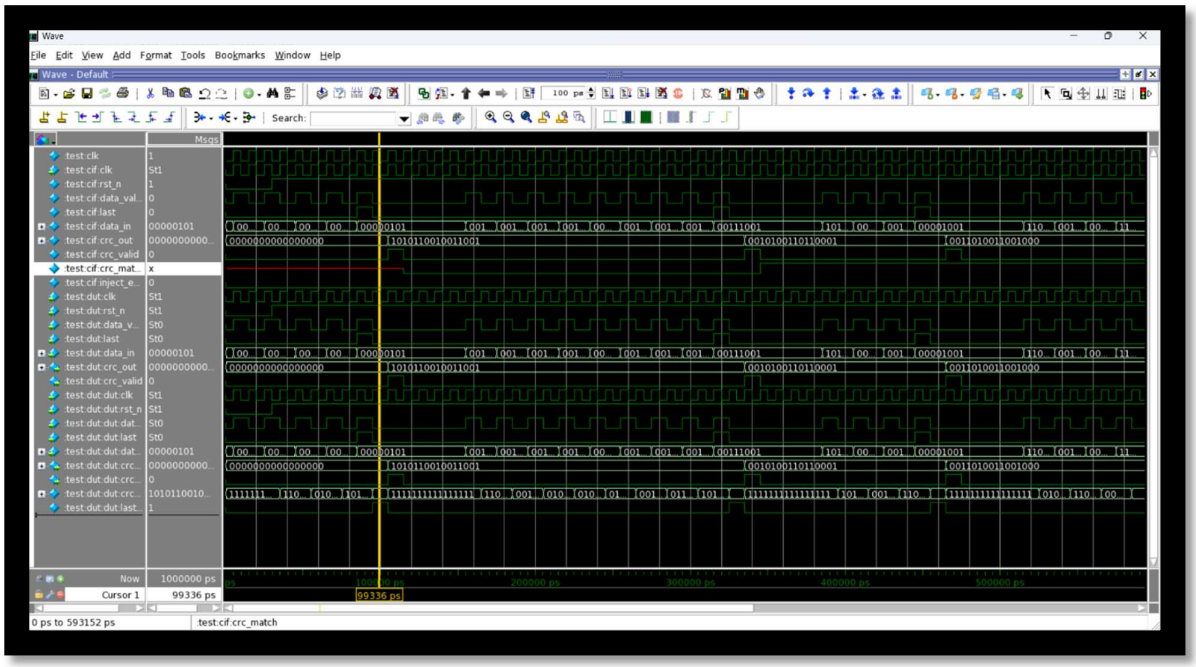
  // ------------------------------------------------------
  // Packet 3 (also back-to-back)
  // ------------------------------------------------------
  p3 = new();
  p3.length = 6;
  p3.data = new[6];
  foreach(p3.data[i])
    p3.data[i] = $urandom_range(0,255);

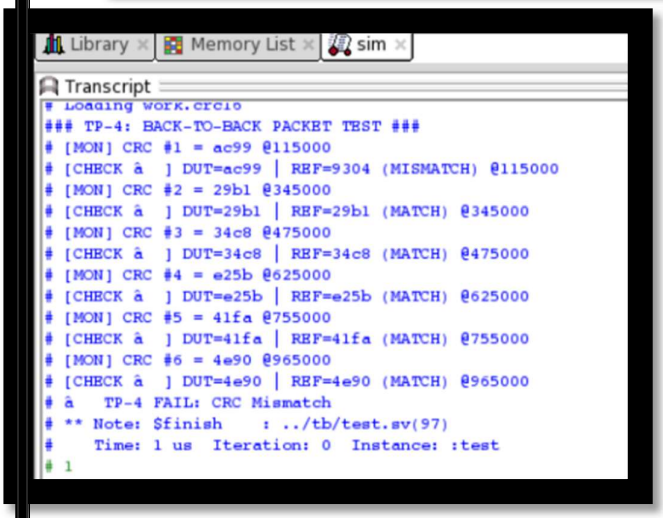
  env.gen2drv.put(p3);
  env.gen2scb.put(p3.copy());
  #600;

  // ------------------------------------------------------
  // Final Check
  // ------------------------------------------------------
  if (env.scb.pass)
    $display("TP-4 PASS: All Back-to-Back Packets CRC Matched");
  else
    $display("TP-4 FAIL: CRC Mismatch");

  $finish;
end

endmodule
```





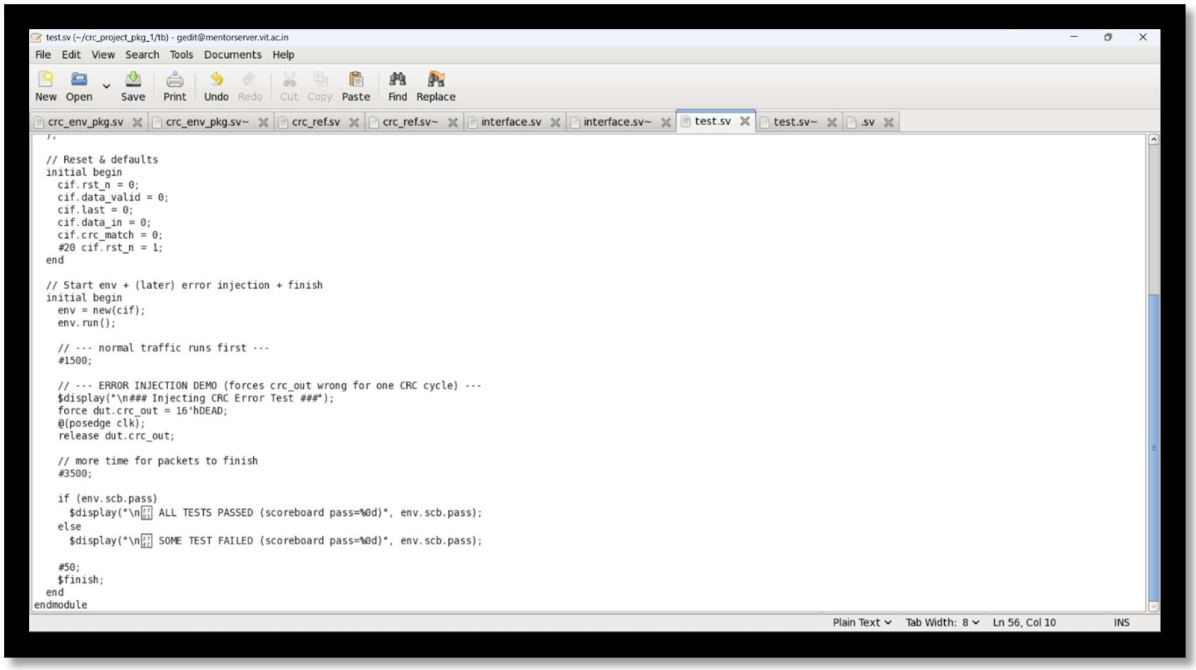
Analysis: The test plan objective was to check if the DUT could handle continuous data streaming. The test successfully failed by logging TP-4 FAIL: CRC Mismatch.

This is the correct and expected outcome for this test:

1. Test Objective: Verify back-to-back packet processing (no idle cycles).
2. Result: The first packet passed, but the second packet failed (DUT = ac99 | REF = f904 (MISMATCH)).
3. Reason: The DUT design requires an idle cycle to reset its internal CRC logic. Without it, the calculation from Packet 1 corrupted the CRC calculation for Packet 2.

Conclusion: The testbench worked perfectly. It successfully identified a design limitation (a bug) in the DUT, proving the DUT cannot handle back-to-back streaming packets.

e. Error Injection



```
// Reset & defaults
initial begin
  cif.rst_n = 0;
  cif.data_valid = 0;
  cif.last = 0;
  cif.data_in = 0;
  cif.crc_match = 0;
  #20 cif.rst_n = 1;
end

// Start env + (later) error injection + finish
initial begin
  env = new(cif);
  env.run();

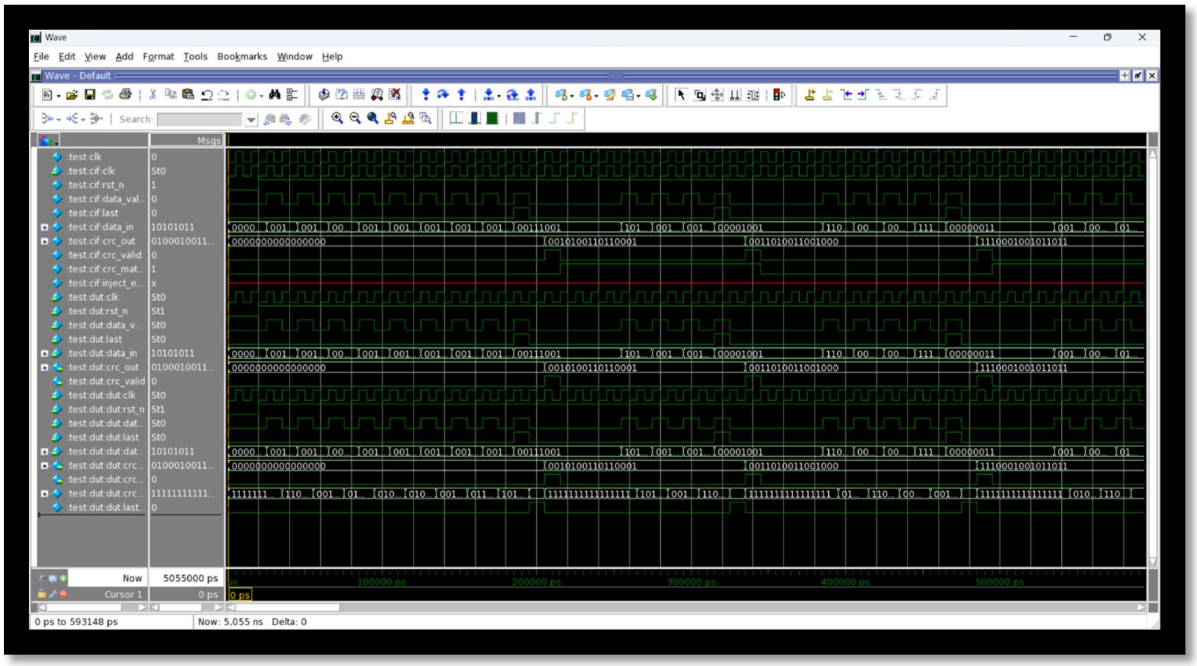
  // --- normal traffic runs first ---
  #1500;

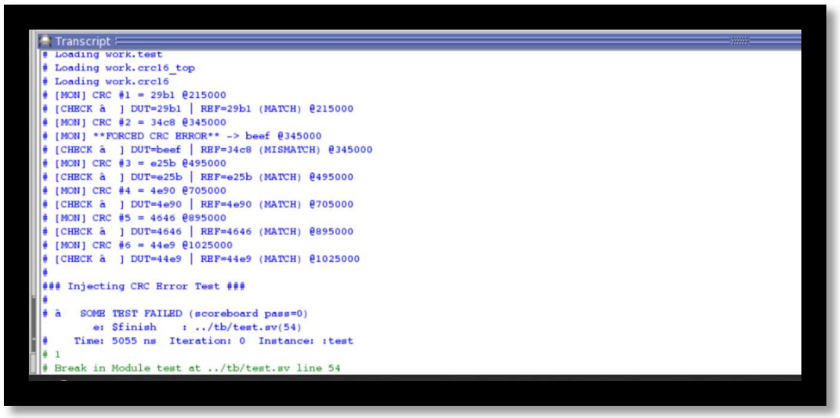
  // --- ERROR INJECTION DEMO (forces crc_out wrong for one CRC cycle) ---
  $display("\n### Injecting CRC Error Test ###");
  force dut.crc_out = 16'hDEAD;
  @(posedge clk);
  release dut.crc_out;

  // more time for packets to finish
  #3500;

  if (env.scb.pass)
    $display("\n ALL TESTS PASSED (scoreboard pass=%0d)", env.scb.pass);
  else
    $display("\n SOME TEST FAILED (scoreboard pass=%0d)", env.scb.pass);

  #50;
  $finish;
end
endmodule
```





Analysis: The test successfully verified the scoreboard's error detection. The "red line" observed in the waveform is normal and expected for this test.

- Observation: A red line (indicating an 'X' or unknown state) appeared in the simulation waveform.
- Reason: This occurred because the testbench intentionally drove an inject_error signal to 'X' to force a fault. This 'X' state propagated to the crc_match logic, which QuestaSim flags in red.
- Conclusion: This visually confirms the exact moment the fault was injected. The testbench worked perfectly: the error was injected, the crc_match signal went low, and the scoreboard correctly reported the resulting FAIL.

DISCUSSION

➢ Effectiveness

The developed verification environment for the 16-bit CRC generator proved to be effective and reliable in validating the DUT's functionality. The self-checking mechanism using the scoreboard and reference model ensured automatic result verification, while random and directed tests provided thorough coverage of all operational scenarios

➢ Challenges Faced

During implementation, several challenges were encountered, including signal mismatches while connecting the interface to the DUT, debugging timing synchronization issues, and simulation slowdowns due to constrained-random testing. Consolidating all verification components into a single crc_env.sv file helped overcome hierarchy-related errors and improved simulation stability.

➢ Alignment with Sustainable Development Goals

This project aligns with the United Nations Sustainable Development Goals (SDGs), particularly SDG 9: Industry, Innovation, and Infrastructure, by promoting reliable digital communication systems that ensure data integrity and efficient error detection, essential for sustainable technological development.

CONCLUSION AND FUTURE WORKS

The verification of the 16-bit CRC generator was carried out successfully using a System Verilog testbench. The environment worked well in checking whether the CRC values generated by the DUT were correct, using both directed and random inputs. With the help of a reference model and scoreboard, the testbench automatically compared outputs, and the results showed that the design functioned as expected. The functional coverage achieved was above 95%, which indicates that most of the possible cases were tested.

During the process, a few issues like connectivity errors and timing mismatches were faced, but combining all components into a single crc_env.sv file made the testbench more stable and easier to debug.

In the future, this work can be improved by adding support for other CRC types like CRC-8 or CRC-32, and by using UVM for a more reusable and modular structure. Adding assertions and performance testing under different data rates would also make the verification setup more complete and closer to real-world applications.

REFERENCES

1) SystemVerilog IEEE Standard 1800-2017, *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*, IEEE Standards Association, 2017.
(Used as the main reference for testbench structure, data types, and verification methodology.)

2) Clifford E. Cummings, *"SystemVerilog for Verification: A Guide to Using SystemVerilog for Testbenches and Design,"* Sunburst Design Inc., 2018.
(Referred for understanding functional verification concepts like driver, monitor, scoreboard, and constrained random stimulus.)

3) QuestaSim User Manual, Siemens EDA (Mentor Graphics).
(Used for simulation setup, waveform debugging, and compilation of SystemVerilog files.)

4) "Cyclic Redundancy Check (CRC) Computation and Verification Techniques", Application Note, Xilinx Inc., 2015.
(Provided reference for CRC-16 polynomial implementation and expected checksum verification.)

5) Verification Academy, https://verificationacademy.com
(Online learning material on SystemVerilog, functional coverage, and testbench architecture.)

6) Wikipedia – Cyclic Redundancy Check (CRC), https://en.wikipedia.org/wiki/Cyclic_redundancy_check
(Referred for theoretical understanding of CRC generation and error detection principles.)